KNOWLEDGEHUB · ARTICLESCONTEXT REFACTOR

How article data reached each component, before and after

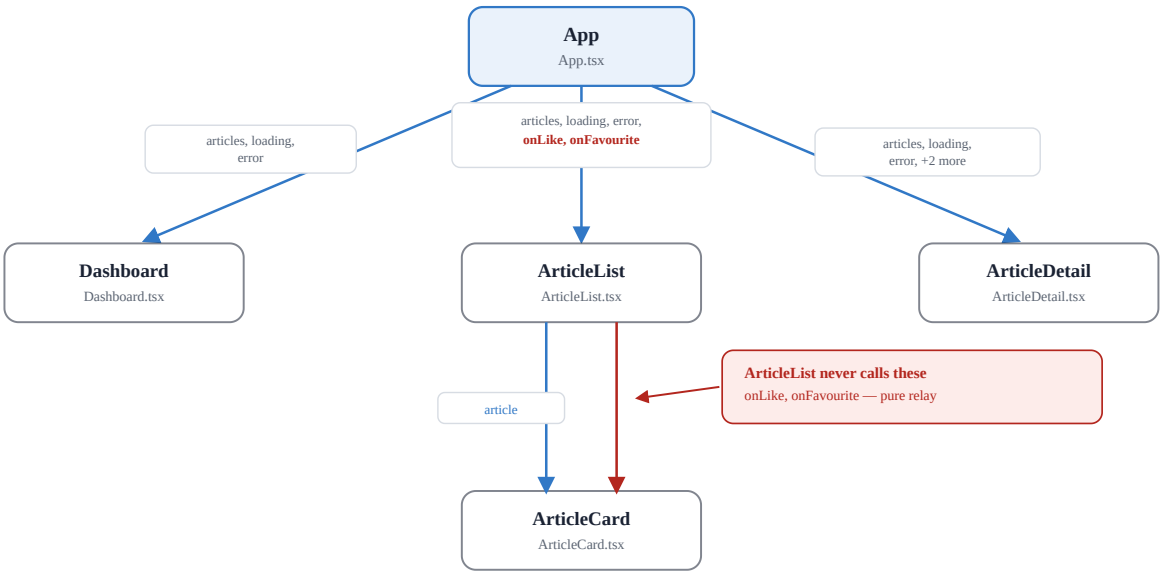
Two views of the same component tree. Before: `App.tsx` owns `articles` / `loading` / `error` and hands them down as props, route by route. After: an `ArticlesProvider` holds that state instead, and each component reads it directly with `useArticles()` — no matter how deep it sits.

prop passed directly by a parent — renders a child — no article data passed

received, but never used here — pure pass-through --- read directly via `useArticles()`

Before — props threaded through the tree

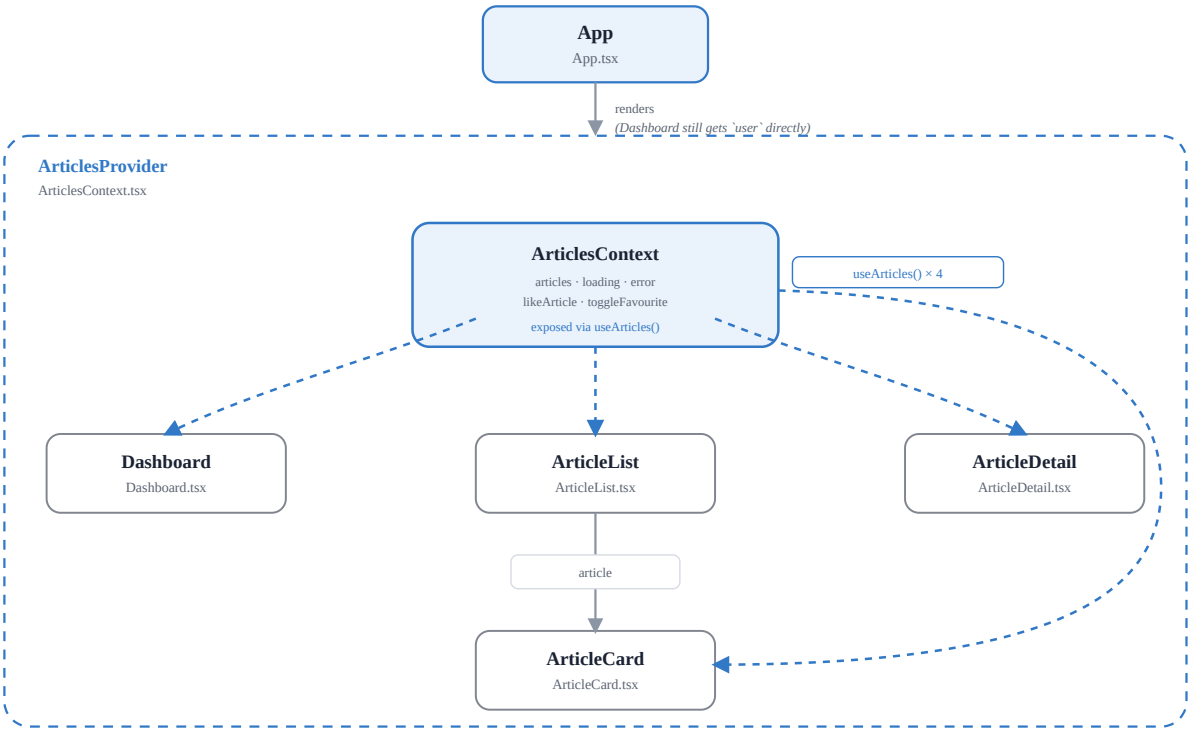
prop drilling



`App.tsx` owns the article state and passes it into every route by hand. `ArticleList` receives `onLike` and `onFavourite` only to relay them to `ArticleCard` — it never calls either one itself.

After — shared via ArticlesContext

`context`



`App.tsx` only renders the tree now — it no longer owns article state. `ArticlesProvider` holds it, and every consumer reads it straight from context. `ArticleList` passes only `article` down to `ArticleCard`; `onLike`/`onFavourite` don't exist as props anywhere in this tree anymore.

What actually changed: five props (`articles`, `loading`, `error`, `onLike`, `onFavourite`) collapsed into one `useArticles()` call wherever they're needed, and the one prop that was pure dead weight — `onLike`/`onFavourite` sitting unused in `ArticleList` — is gone rather than just relocated.